॥ वसुधैव कुटुम्बकम् ॥

SYMBIOSIS INTERNATIONAL (DEEMED UNIVERSITY)

(Established under section 3 of the UGC Act 1956)

Re-accredited by NAAC with "A" Grade

Name of the Student: Vatsal Postiwala **PRN: 26070122509**

Semester: III **Year: 2025-29**

Subject Title: Operating Systems Lab

EXPERIMENT No: 11

TITLE: Banker's Algorithm **DoP**: 05/10/26

Aim: Implement a C program to demonstrate the Banker's algorithm.

Learning Outcome: 1. To understand the deadlock concept
2. To Demonstrate the Banker's algorithm

Hardware/Software:

- Hardware requirements:
  - A PC with a minimum of 4GB RAM.
  - A processor with 1.6GHz clock speed or faster.
  - Sufficient hard disk space to store the code and compiler.
- Software Requirements:
  - Visual Studio Code (VS Code)
  - C++ compiler such as GCC
  - C/C++ extension for VS Code. This extension compiles and runs C++ code directly from VS Code.

Theory:

Banker's algorithm is a resource allocation and deadlock avoidance algorithm used in operating systems. It is used to ensure that a system will not enter a deadlock state while allocating resources to multiple processes. The algorithm works by simulating the allocation of resources to processes and checking whether the system can reach a safe state, i.e., a state where no deadlock can occur.

The algorithm consists of multiple steps:

- Initially, the system has a fixed number of resources of each type, and each process has a maximum demand for each resource type.
- When a process requests resources, the system checks whether the request can be granted without violating the safety constraints.
- If the request can be granted, the system simulates the allocation of resources to the process

and checks whether the resulting state is safe.

- If the resulting state is safe, the resources are allocated to the process. Otherwise, the request is denied, and the process must wait until resources become available.

The features of the Banker's algorithm include:

- Deadlock avoidance: The Banker's algorithm is designed to prevent the occurrence of a deadlock state by ensuring that the system will always be able to allocate resources to at least one process.
- Resource allocation: The Banker's algorithm ensures that resources are allocated in a safe and efficient manner, without causing any unnecessary delays or conflicts.
- Predictable behavior: The Banker's algorithm ensures that the behavior of the system is predictable and deterministic, as the algorithm uses a fixed set of rules to allocate resources.

The drawbacks of the Banker's algorithm include:

- Overhead: The Banker's algorithm requires additional overhead to maintain the state of the system and to simulate the allocation of resources to processes.
- Restrictive: The Banker's algorithm can be restrictive in some cases, as it may deny requests for resources even if they are not causing any conflicts or delays.
- Limitations on resource allocation: The Banker's algorithm may limit the number of resources that can be allocated to a process, which can affect the performance of the system in certain scenarios.

Problem Statement:

1. A system has **5 processes P0–P4** and **3 resource types A, B, and C**.

The following information is given:

(current Available = (3, 3, 2)

Allocation Matrix:

Process	**A**	**B**	**C**
P0	0	1	0
P1	2	0	0
P2	3	0	2
P3	2	1	1
P4	0	0	2

Max matrix:

Process	**A**	**B**	**C**
P0	7	5	3
P1	3	2	2
P2	9	0	2
P3	2	2	2

P4	4	3	3

PRN :- 2607012250 9

Name :- Posticwala Vatsal Dipakbhai

PAGE NO.
DATE:

★ Problem I :-

Available = (3, 3, 2)

Process	MAX			Allocation			Need			Available		
	A	B	C	A	B	C	A	B	C	A	B	C
P0	7	5	3	0	1	0	7	4	3	7	5	5
P1	3	2	2	2	0	0	1	2	2	5	3	2
P2	9	0	2	3	0	2	6	0	0	10	5	7
P3	2	2	2	2	1	1	0	1	1	7	4	3
P4	4	3	3	0	0	2	4	3	1	7	4	5

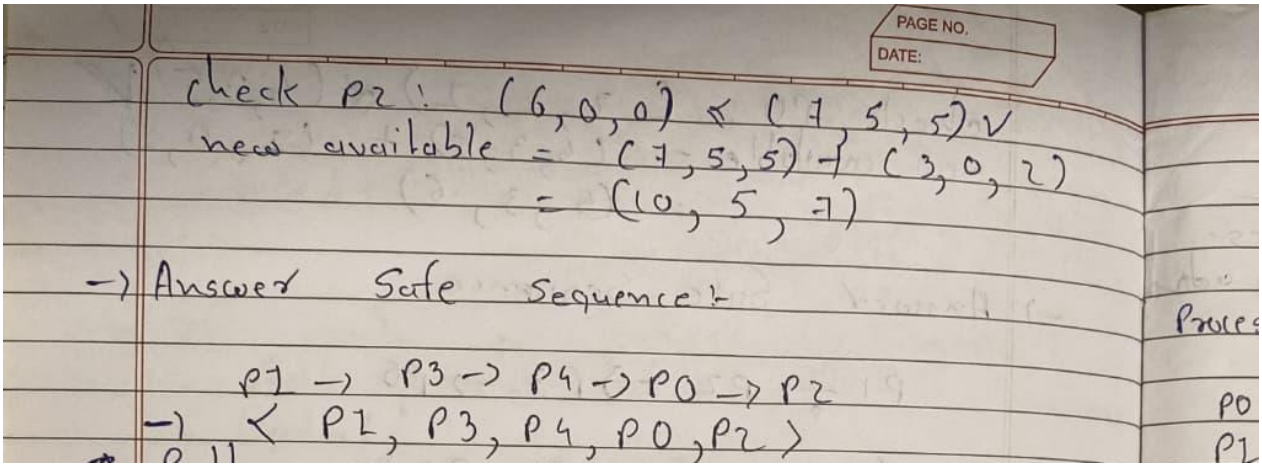
⇒ Now we Compare Need & Available

- check P0 : (7, 4, 3) ≤ (3, 3, 2) ✗
- check P1 : (1, 2, 2) ≤ (3, 3, 2) ✓
  ∴ new available = available + P1 (Allocation)
  = (3, 3, 2) + (2, 0, 0)
  = (5, 3, 2)

- check P2 : (6, 0, 0) ≤ (5, 3, 2) ✗
- check P3 : (0, 1, 1) ≤ (5, 3, 2) ✓
  new available = (5, 3, 2) + (2, 1, 1)
  = (7, 4, 3)

- check P4 : (4, 3, 1) ≤ (7, 4, 3) ✓
  new available = (7, 4, 3) + (0, 0, 2)
  = (7, 4, 5)

- check P0 : (7, 4, 3) ≤ (7, 4, 5) ✓
  new available = (7, 4, 5) + (0, 1, 0)
  = (7, 5, 5)



2. A system consists of **4 processes P0–P3** and **3 resource types X, Y, Z**.

Current Available = (1,\1, 1)
Allocation Matrix:

Process	X	Y	Z
P0	1	0	1
P1	1	1	2
P2	2	1	0
P3	0	0	2

Max Matrix:

Process	X	Y	Z
P0	3	2	2
P1	2	2	3
P2	4	2	1
P3	2	2	2

PAGE NO.
DATE:

check P2 : (6, 0, 0) ≤ (7, 5, 5) ✓
new available = (7, 5, 5) + (3, 0, 2)
= (10, 5, 7)

→ Answer Safe Sequence :-

P1 → P3 → P4 → P0 → P2
→ < P1, P3, P4, P0, P2 >

✗ Problem 2 :-

Available = (1, 2, 2)

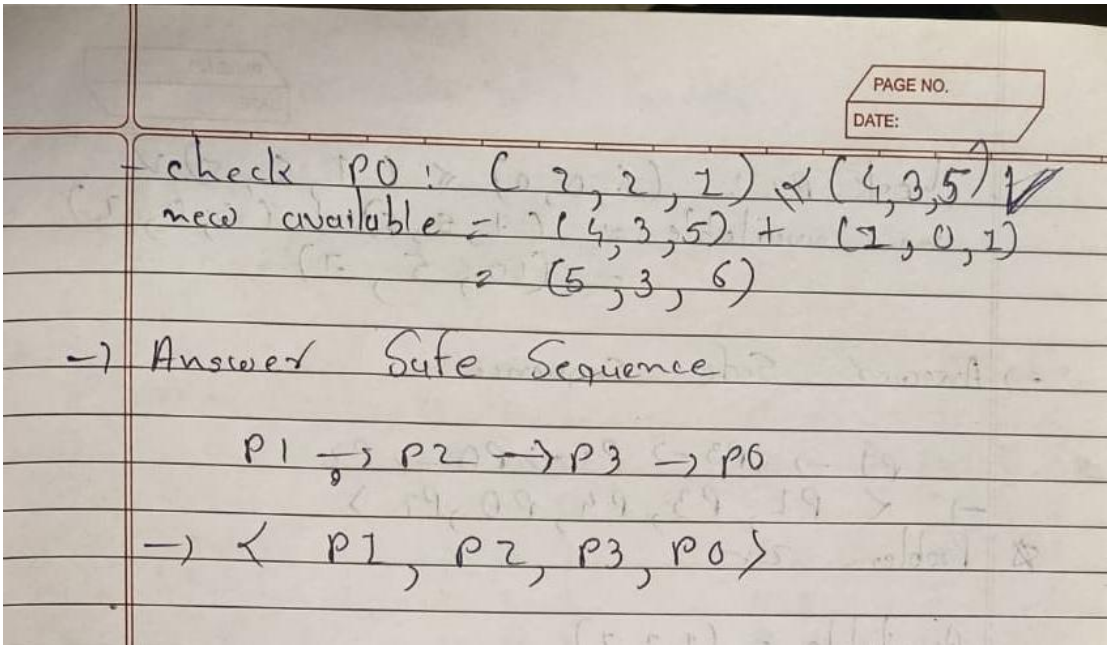
Process	MAX	Allocation	Need	Available
	X Y Z	X Y Z	X Y Z	X Y Z
P0	3 2 2	1 0 1	2 2 1	5 3 6
P1	2 2 3	1 2 2	1 1 1	2 2 3
P2	4 2 1	2 1 0	2 1 1	4 3 3
P3	2 2 2	0 0 2	2 2 0	4 3 5

=) Find the Sequence :-

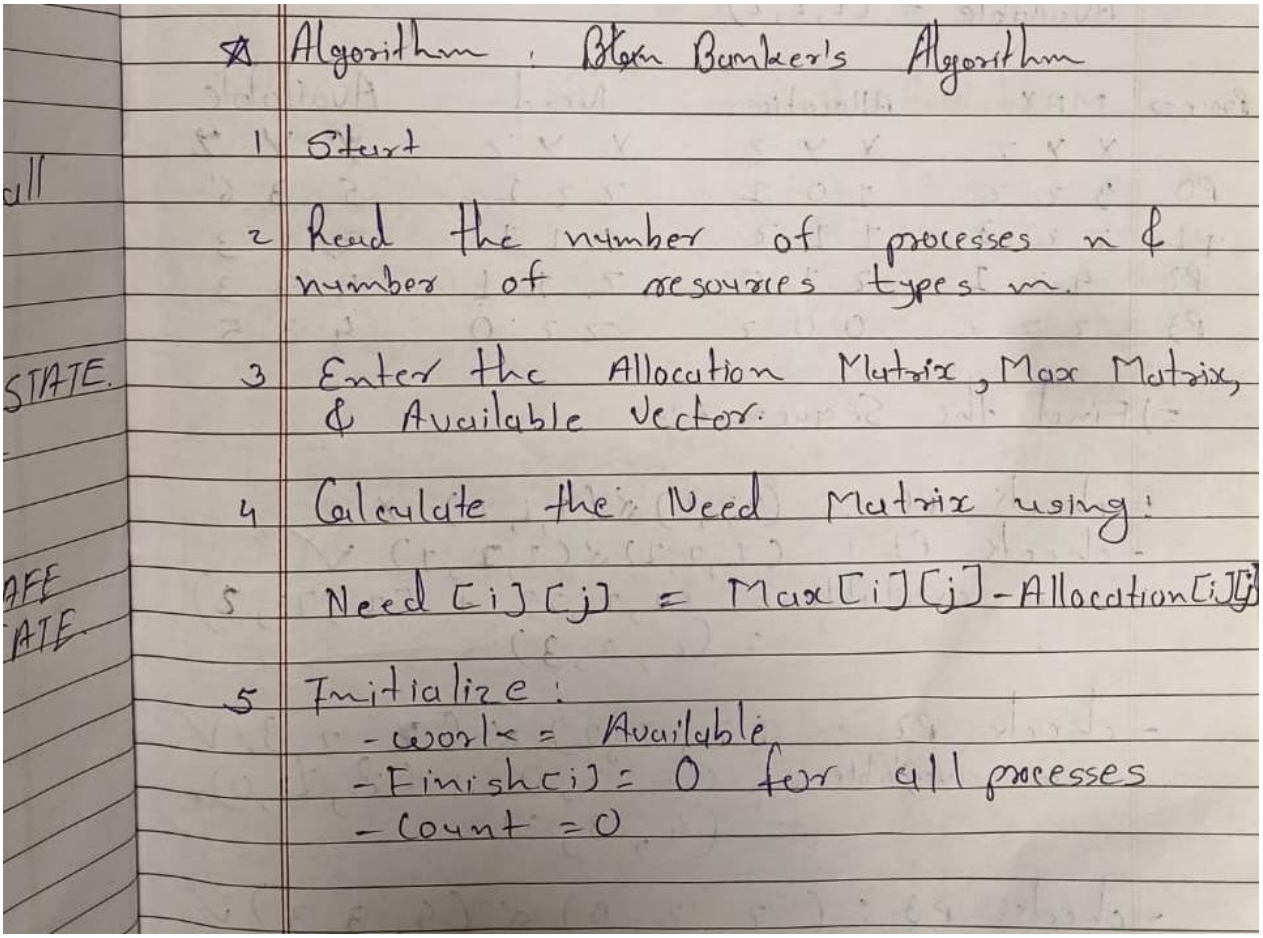
- check P0 : (2, 2, 2) ≤ (1, 2, 2) ✗
- check P1 : (1, 1, 2) ≤ (1, 2, 2) ✓
new available = (1, 2, 2) + (1, 2, 2)
= (2, 3, 3)

- check P2 : (2, 1, 2) ≤ (2, 3, 3) ✓
new available = (2, 3, 3) + (2, 1, 0)
= (4, 3, 3)

- check P3 : (2, 2, 0) ≤ (4, 3, 3) ✓
new available = (4, 3, 3) + (0, 0, 2)
= (4, 3, 5)


PAGE NO.
DATE:
- check P0 : (2,2,1) ≤ (4,3,5)
new available = (4,3,5) + (2,0,1)
= (5,3,6)
→ Answer Safe Sequence
P1 → P2 → P3 → P0
→ < P1, P2, P3, P0 >

Algorithm:


☆ Algorithm : Banker's Algorithm
1 Start
2 Read the number of processes n & number of resources types m.
3 Enter the Allocation Matrix, Max Matrix, & Available Vector.
4 Calculate the Need Matrix using:
5 Need[i][j] = Max[i][j] - Allocation[i][j]
5 Initialize:
- work = Available
- Finish[i] = 0 for all processes
- count = 0

PAGE NO.
DATE:

6 Select a process Pi whose:
- Finish[i] = 0
- Need[i][j] <= work[j] for every resource j.

7 If such a process is found:
- Add process Pi to the Safe Sequence
- Update:
  work[j] = work[j] + Allocation[i][j]
- set:
  Finish[i] = 1
- Increment Count.

8 Repeate Steps 6 & 7 untill all processes are completed.

9 If Count = n, then:
- The system is in a SAFE STATE.
- Display the Safe Sequence

10 If Count != n, then:
- The System is in an UNSAFE STATE.
- Deadlock may occure.

11 Stop.

Program:

Code 1 :-

```
#include <stdio.h>
#include <stdbool.h>

int main() {
    int n = 5;
    int m = 3;

    int alloc[5][3] = {
        {0, 1, 0},
        {2, 0, 0},
        {3, 0, 2},
        {2, 1, 1},
        {0, 0, 2}
    };

    int max[5][3] = {
        {7, 5, 3},
        {3, 2, 2},
        {9, 0, 2},
        {2, 2, 2},
        {4, 3, 3}
    };

    int avail[3] = {3, 3, 2};
    int need[5][3], finish[5], safeSeq[5], work[3];
    int count = 0;

    for (int i = 0; i < n; i++) {
        finish[i] = 0;
        for (int j = 0; j < m; j++) {
            need[i][j] = max[i][j] - alloc[i][j];
        }
    }

    printf("--- Problem 1 Execution ---\n");
    printf("Initial Available: %d %d %d\n\n", avail[0], avail[1], avail[2]);

    for (int i = 0; i < m; i++) {
        work[i] = avail[i];
    }

    while (count < n) {
        bool found = false;
        for (int p = 0; p < n; p++) {
            if (finish[p] == 0) {
                int j;
                for (j = 0; j < m; j++) {
```

```
                if (need[p][j] > work[j]) {
                    break;
                }
            }
            if (j == m) {
                printf("Process P%d executes. ", p);
                for (int k = 0; k < m; k++) {
                    work[k] += alloc[p][k];
                }

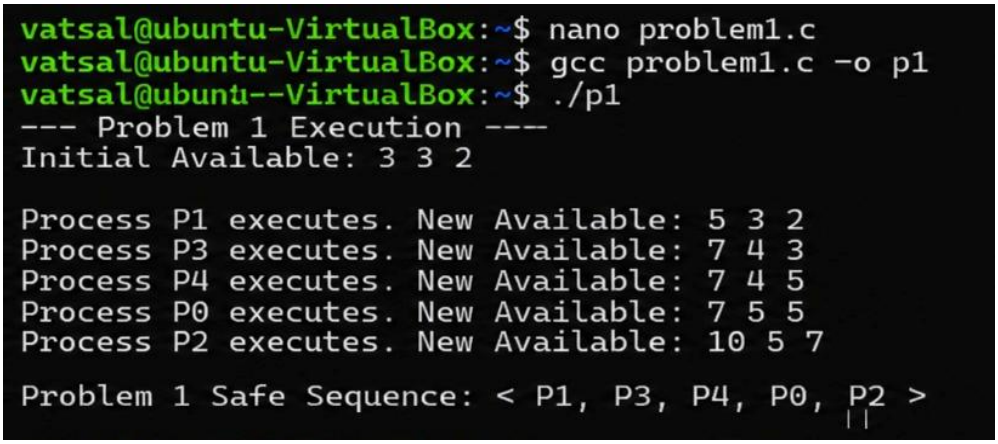
                // Print the updated Available (Work) array
                printf("New Available: ");
                for (int k = 0; k < m; k++) {
                    printf("%d ", work[k]);
                }
                printf("\n");

                safeSeq[count++] = p;
                finish[p] = 1;
                found = true;
            }
        }
    }
    if (found == false) {
        printf("\nSystem is NOT in a safe state.\n");
        return 0;
    }
}

printf("\nProblem 1 Safe Sequence: < ");
for (int i = 0; i < n; i++) {
    printf("P%d%s", safeSeq[i], (i == n - 1) ? " >\n" : ", ");
}

return 0;
}
```

Output :-



Code 2 :-

```
#include <stdio.h>
#include <stdbool.h>

int main() {
    int n = 4;
    int m = 3;

    int alloc[4][3] = {
        {1, 0, 1},
        {1, 1, 2},
        {2, 1, 0},
        {0, 0, 2}
    };

    int max[4][3] = {
        {3, 2, 2},
        {2, 2, 3},
        {4, 2, 1},
        {2, 2, 2}
    };

    int avail[3] = {1, 1, 1};
    int need[4][3], finish[4], safeSeq[4], work[3];
    int count = 0;

    for (int i = 0; i < n; i++) {
        finish[i] = 0;
        for (int j = 0; j < m; j++) {
            need[i][j] = max[i][j] - alloc[i][j];
        }
    }

    printf("--- Problem 2 Execution ---\n");
    printf("Initial Available: %d %d %d\n\n", avail[0], avail[1], avail[2]);

    for (int i = 0; i < m; i++) {
        work[i] = avail[i];
    }

    while (count < n) {
        bool found = false;
        for (int p = 0; p < n; p++) {
            if (finish[p] == 0) {
                int j;
                for (j = 0; j < m; j++) {
                    if (need[p][j] > work[j]) {
                        break;
                    }
                }
```

```
            if (j == m) {
                printf("Process P%d executes. ", p);
                for (int k = 0; k < m; k++) {
                    work[k] += alloc[p][k];
                }

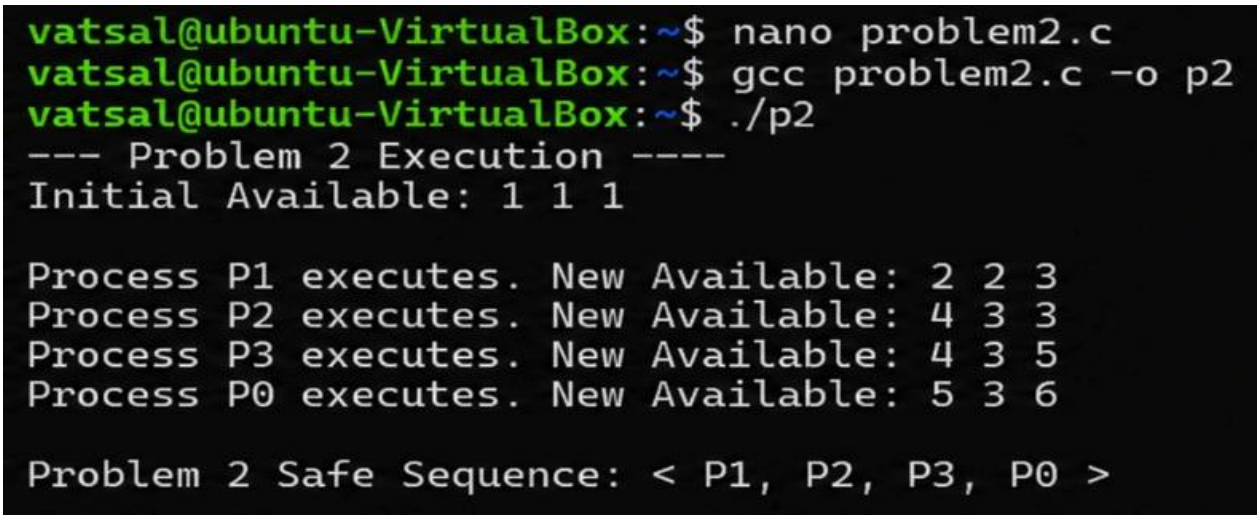
                // Print the updated Available (Work) array
                printf("New Available: ");
                for (int k = 0; k < m; k++) {
                    printf("%d ", work[k]);
                }
                printf("\n");

                safeSeq[count++] = p;
                finish[p] = 1;
                found = true;
            }
        }
    }
    if (found == false) {
        printf("\nSystem is NOT in a safe state.\n");
        return 0;
    }
}

printf("\nProblem 2 Safe Sequence: < ");
for (int i = 0; i < n; i++) {
    printf("P%d%s", safeSeq[i], (i == n - 1) ? " >\n" : ", ");
}

return 0;
}
```

Output :-



Conclusion:

Through this experiment, the Banker's algorithm was successfully implemented in C to demonstrate deadlock avoidance. By computing the 'Need' matrix (Max - Allocation) and simulating the stepwise allocation of resources, the programs successfully evaluated the given system states. It verified whether a safe state existed and, when possible, generated the correct safe sequence of process execution without entering a deadlock. This practical implementation solidified the understanding of how operating systems proactively prevent deadlocks by granting resource requests only if the resulting state is mathematically proven to be safe.

Assessment Schemes:

Attendance	Discipline	Short oral	Correctness of Lab Report	Timely completion of Lab Report	Total marks (10)	Signature of Teacher with Date